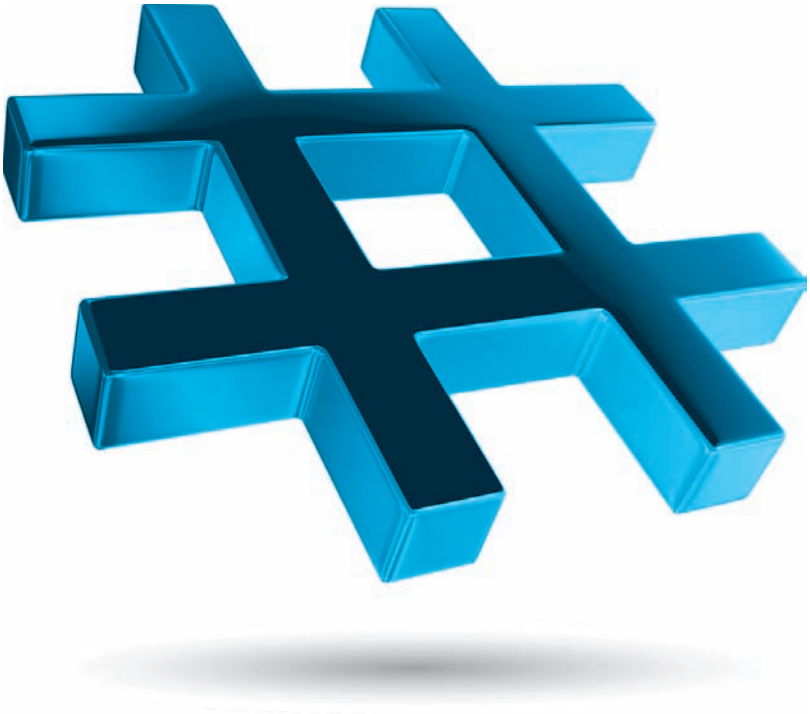
Introduction to Visual Studio and Visual Programming

2



Objectives

In this chapter you'll:

- Learn the basics of the Visual Studio Community 2015 Integrated Development Environment (IDE) for writing, running and debugging your apps.
- Create a new project using Visual Studio's **Windows Forms Application** template.
- Be introduced to Windows `Forms` and the controls you'll use to build graphical user interfaces.
- Use key commands contained in the IDE's menus and toolbars.
- Understand the purpose of the various windows in the Visual Studio.
- Use Visual Studio's help features.
- Use visual app development to conveniently create, compile and execute a simple Visual C# app that displays text and an image.

Outline

2.1 Introduction
2.2 Overview of the Visual Studio Community 2015 IDE
2.2.1 Introduction to Visual Studio Community 2015
2.2.2 Visual Studio Themes
2.2.3 Links on the Start Page
2.2.4 Creating a New Project
2.2.5 **New Project** Dialog and Project Templates
2.2.6 Forms and Controls
2.3 Menu Bar and Toolbar
2.4 Navigating the Visual Studio IDE
2.4.1 **Solution Explorer**
2.4.2 **Toolbox**
2.4.3 **Properties** Window
2.5 Help Menu and Context-Sensitive Help
2.6 Visual Programming: Creating a Simple App that Displays Text and an Image
2.7 Wrap-Up
2.8 Web Resources

Summary | *Terminology* | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

2.1 Introduction

Visual Studio is Microsoft's Integrated Development Environment (IDE) for creating, running and debugging apps (also called applications) written in C# and various other .NET programming languages. In this chapter, we overview the Visual Studio Community 2015 IDE, then show how to create a simple Visual C# app by dragging and dropping predefined building blocks into place—a technique known as visual app development.

2.2 Overview of the Visual Studio Community 2015 IDE

There are several versions of Visual Studio. This book's examples, screen captures and discussions are based on the free Visual Studio Community 2015 running on Windows 10. See the Before You Begin section that follows the Preface for information on installing the software. With few exceptions, this book's examples can be created and run on Windows 7, 8.x or 10—we'll point out any examples that require Windows 10.

The examples will work on full versions of Visual Studio as well—though some options, menus and instructions might differ. From this point forward, we'll refer to Visual Studio Community 2015 simply as "Visual Studio" or "the IDE." We assume that you have some familiarity with Windows.

2.2.1 Introduction to Visual Studio Community 2015

[*Note:* We use the **>** character to indicate when you should select a *menu item* from a *menu*. For example, notation **File > Save All** means that you should select the **Save All** menu item from the **File** menu.]

To begin, open Visual Studio. On Windows 10, click



then select **All Apps > Visual Studio 2015**. On Windows 7, click



then select **All Programs > Visual Studio 2015**. On Windows 8's **Start** screen, locate and click the Visual Studio 2015 tile, which will contain the following icon:



Initially, Visual Studio displays the **Start Page** (Fig. 2.1). Depending on your version of Visual Studio, your **Start Page** may look different. The **Start Page** contains a list of links to Visual Studio resources and web-based resources. At any time, you can return to the **Start Page** by selecting **View > Start Page**.

2.2.2 Visual Studio Themes

Visual Studio supports three themes that specify the IDE's color scheme:

- a *dark theme* (with dark window backgrounds and light text)
- a *light theme* (with light window backgrounds and dark text) and
- a *blue theme* (with light window backgrounds and dark text).

We use the blue theme throughout this book. The Before You Begin section after the Preface explains how to set this option.

2.2.3 Links on the Start Page

The **Start Page** links are organized into two columns. The left column's **Start** section contains options for building new apps or working on existing ones. The left column's **Recent** section contains links to projects you've recently created or modified.

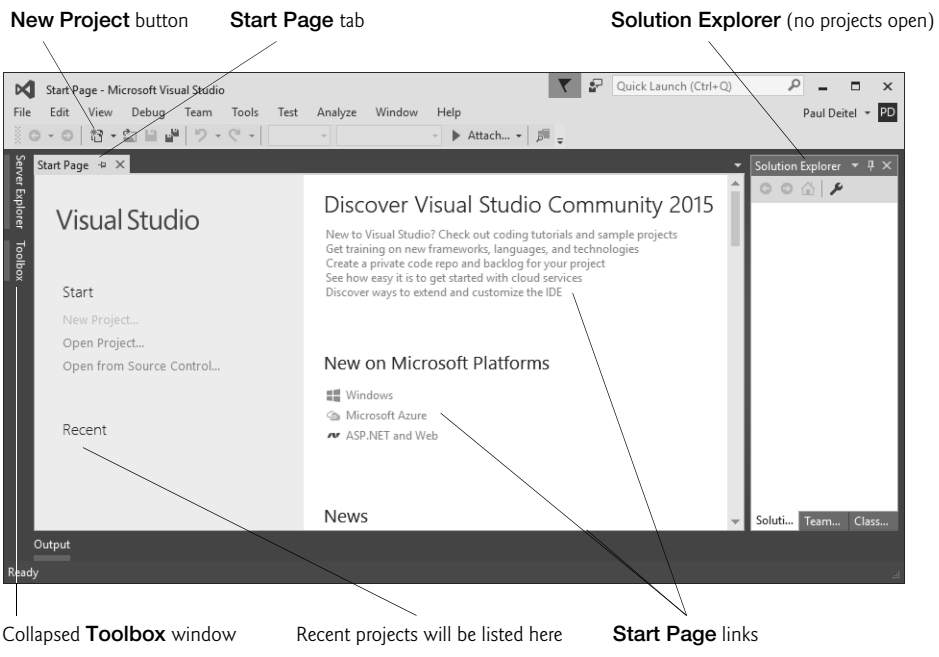


Fig. 2.1 | **Start Page** in Visual Studio Community 2015.

The **Start Page**'s right column—with **Discover Visual Studio Community 2015** at the top—contains links to various online documentation and resources to help you get started with Visual Studio and learn about Microsoft programming technologies. An Internet connection is required for the IDE to access most of this information.

To access more extensive information on Visual Studio, you can browse the MSDN (Microsoft Developer Network) Library at

```
https://msdn.microsoft.com/library/dd831853
```

The MSDN site contains articles, downloads and tutorials on technologies of interest to Visual Studio developers. You also can browse the web from the IDE by selecting **View > Other Windows > Web Browser**. To request a web page, type its URL into the location bar (Fig. 2.2) and press the *Enter* key—your computer, of course, must be connected to the Internet. The web page that you wish to view appears as another tab in the IDE—Figure 2.2 shows the browser tab after entering `http://msdn.microsoft.com/library`.

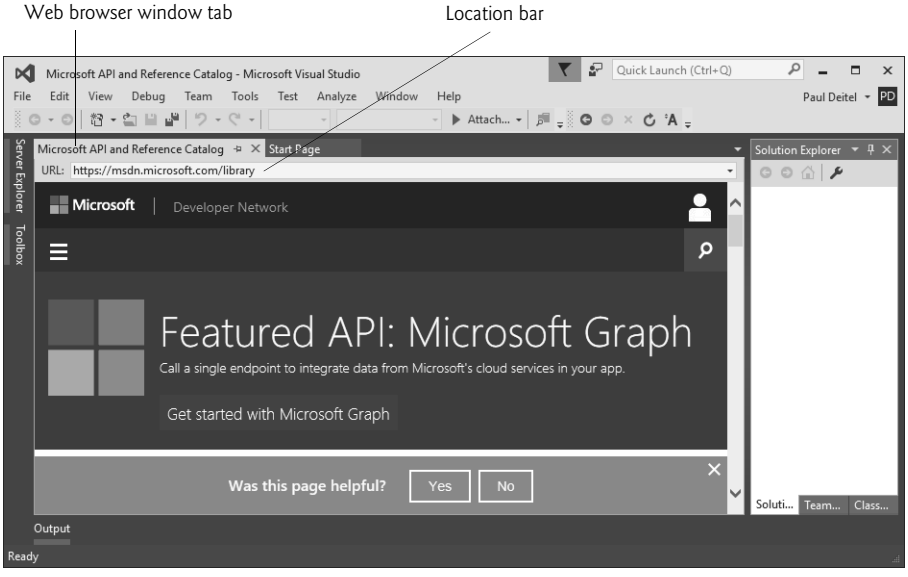


Fig. 2.2 | MSDN Library web page in Visual Studio.

2.2.4 Creating a New Project

To begin app development in Visual C#, you must create a new project or open an existing one. A **project** is a group of related files, such as the Visual C# code and any images that might make up an app. Visual Studio organizes apps into projects and **solutions**, which contain one or more projects. Multiple-project solutions are used to create large-scale apps. Most apps we create in this book consist of a solution containing a single project. You select **File > New > Project...** to create a new project or **File > Open > Project/Solution...** to

open an existing one. You also can click the corresponding links in the **Start Page**'s **Start** section.

2.2.5 New Project Dialog and Project Templates

For the discussions in the next several sections, we'll create a new project. Select **File > New > Project...** to display the **New Project** dialog (Fig. 2.3). Dialogs are windows that facilitate user–computer communication.

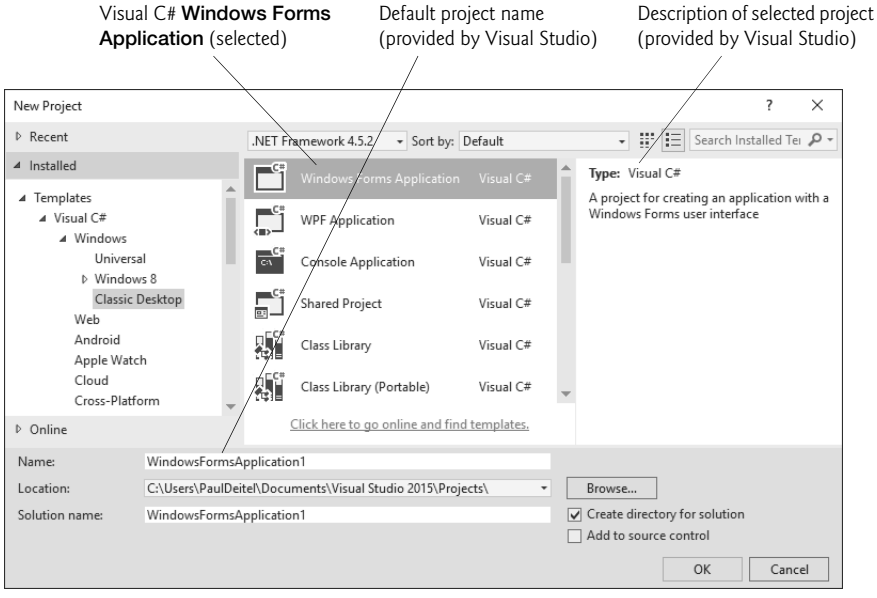


Fig. 2.3 | **New Project** dialog.

Visual Studio provides many **templates** (left column of Fig. 2.3)—the *project types* that users can create in Visual C# and other languages. The templates include Windows Forms apps, WPF apps and others—full versions of Visual Studio provide additional templates. In this chapter, you'll build a **Windows Forms Application**—an app that executes within a Windows operating system (such as Windows 7, 8 or 10) and typically has a **graphical user interface** (**GUI**). Users interact with this *visual* part of the app. GUI apps include Microsoft software products like Microsoft Word, Internet Explorer and Visual Studio, software products created by other vendors, and customized software that you and other app developers create. You'll create many Windows apps in this book.

To create a **Windows Forms Application**, under **Templates** select **Visual C# > Windows > Classic Desktop**, then in the middle column select **Windows Forms Application**. By default, Visual Studio assigns the name **WindowsFormsApplication1** to a new **Windows Forms Application** project and solution (Fig. 2.3). Click **OK** to display the IDE in **Design view** (Fig. 2.4), which contains the features that enable you to create an app's GUI.

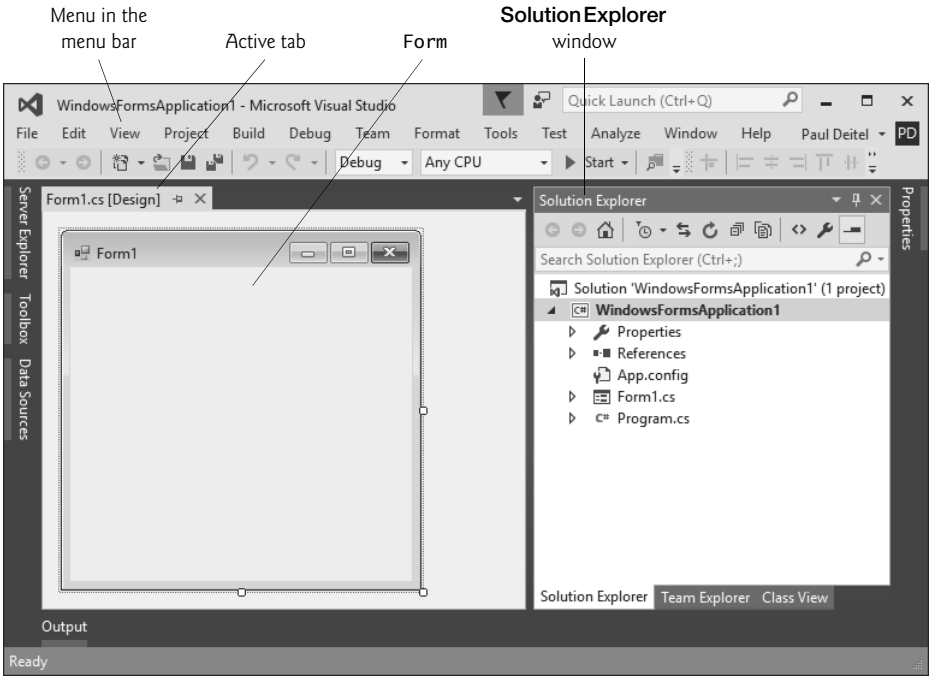


Fig. 2.4 | **Design** view of the IDE.

2.2.6 Forms and Controls

The rectangle in the **Design** area titled **Form1** (called a `Form`) represents the main window of the Windows `Forms` app that you're creating. Each `Form` is an object of class `Form` in the .NET Framework Class Library. Apps can have multiple `Forms` (windows)—however, the app you'll create in Section 2.6 and most other Windows `Forms` app you'll create later in this book will contain a single `Form`. You'll learn how to customize the `Form` by adding GUI **controls**—in Section 2.6, you'll add a `Label` and a `PictureBox`. A `Label` typically contains descriptive text (for example, `"Welcome to Visual C#!"`), and a `PictureBox` displays an image. Visual Studio has many preexisting controls and other components you can use to build and customize your apps. Many of these controls are discussed and used throughout the book. Other controls are available from third parties.

In this chapter, you'll work with preexisting controls from the .NET Framework Class Library. As you place controls on the `Form`, you'll be able to modify their properties (discussed in Section 2.4).

Collectively, the `Form` and controls make up the app's GUI. Users enter data into the app by typing at the keyboard, by clicking the mouse buttons and in a variety of other ways. Apps use the GUI to display instructions and other information for users to view. For example, the **New Project** dialog in Fig. 2.3 presents a GUI where the user clicks the mouse button to select a template type, then inputs a project name from the keyboard (the figure shows the default project name **WindowsFormsApplication1**).

Each open document's name is listed on a tab. To view a document when multiple documents are open, click its tab. The **active tab** (the tab of the currently displayed document) is highlighted (for example, **Form1.cs [Design]** in Fig. 2.4). The active tab's highlight color depends on the Visual Studio theme—the blue theme uses a yellow highlight and the light and dark themes use a blue highlight.

2.3 Menu Bar and Toolbar

Commands for managing the IDE and for developing, maintaining and executing apps are contained in menus, which are located on the **menu bar** of the IDE (Fig. 2.5). The set of menus displayed depends on what you're currently doing in the IDE.

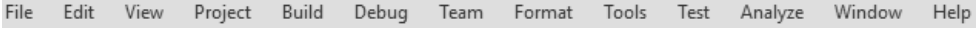


Fig. 2.5 | Visual Studio menu bar.

Menus contain groups of related commands called *menu items* that, when selected, cause the IDE to perform specific actions—for example, open a window, save a file, print a file and execute an app. For example, selecting **File > New > Project...** tells the IDE to display the **New Project** dialog. The menus depicted in Fig. 2.5 are summarized in Fig. 2.6.

Menu	Contains commands for
File	Opening, closing, adding and saving projects, as well as printing project data and exiting Visual Studio.
Edit	Editing apps, such as cut, copy, paste, undo, redo, delete, find and select.
View	Displaying IDE windows (for example, **Solution Explorer**, **Toolbox**, **Properties** window) and for adding toolbars to the IDE.
Project	Managing projects and their files.
Build	Turning your app into an executable program.
Debug	Compiling, debugging (that is, identifying and correcting problems in apps) and running apps.
Team	Connecting to a Team Foundation Server—used by development teams that typically have multiple people working on the same app.
Format	Arranging and modifying a `Form`'s controls. The **Format** menu appears *only* when a GUI component is selected in **Design** view.
Tools	Accessing additional IDE tools and options for customizing the IDE.
Test	Performing various types of automated testing on your app.
Analyze	Locating and reporting violations of the .NET Framework Design Guidelines (`https://msdn.microsoft.com/library/ms229042`).

Fig. 2.6 | Summary of Visual Studio menus that are displayed when a `Form` is in **Design** view. (Part 1 of 2.)

Menu	Contains commands for
Window	Hiding, opening, closing and displaying IDE windows.
Help	Accessing the IDE's help features.

Fig. 2.6 | Summary of Visual Studio menus that are displayed when a `Form` is in **Design** view. (Part 2 of 2.)

You can access many common menu commands from the **toolbar** (Fig. 2.7), which contains **icons** that graphically represent commands. By default, the standard toolbar is displayed when you run Visual Studio for the first time—it contains icons for the most commonly used commands, such as opening a file, saving files and running apps (Fig. 2.7). The icons that appear on the standard toolbar may vary, depending on the version of Visual Studio you're using. Some commands are initially disabled (grayed out or unavailable to use). These commands are enabled by Visual Studio only when you can use them. For example, Visual Studio enables the command for saving a file once you begin editing a file.

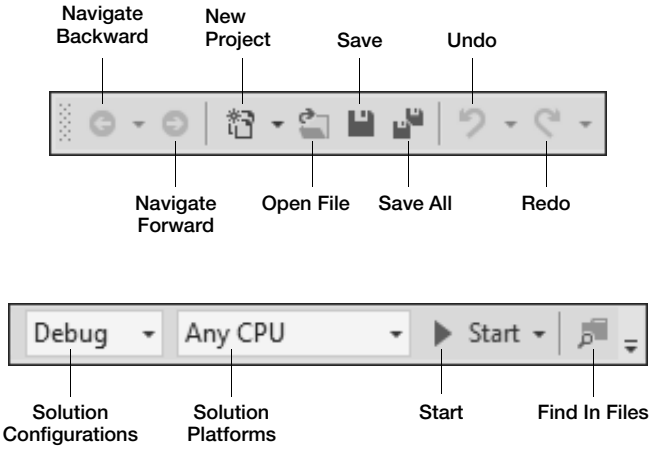


Fig. 2.7 | Standard Visual Studio toolbar.

You can customize which toolbars are displayed by selecting **View > Toolbars** then selecting a toolbar from the list in Fig. 2.8. Each toolbar you select is displayed with the other toolbars at the top of the Visual Studio window. You move a toolbar by dragging its handle



at the left side of the toolbar. To execute a command via the toolbar, click its icon.

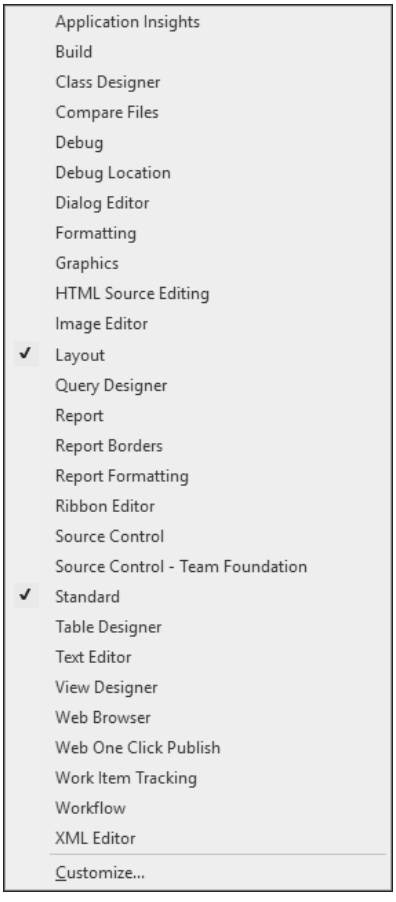


Fig. 2.8 | List of toolbars that can be added to the top of the IDE.

It can be difficult to remember what each toolbar icon represents. *Hovering* the mouse pointer over an icon highlights it and, after a brief pause, displays a description of the icon called a **tool tip** (Fig. 2.9)—these tips help you become familiar with the IDE's features and serve as useful reminders for each toolbar icon's functionality.

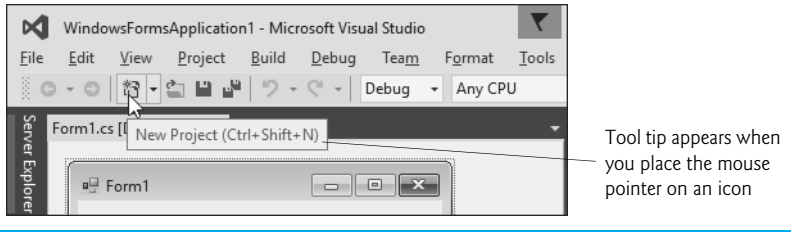


Fig. 2.9 | Tool tip for the **New Project** button.

2.4 Navigating the Visual Studio IDE

The IDE provides windows for accessing project files and customizing controls. This section introduces several windows that you'll use frequently when developing Visual C# apps. Each of the IDE's windows can be accessed by selecting its name in the **View** menu.

Auto-Hide

Visual Studio provides an **auto-hide** space-saving feature. When auto-hide is enabled for a window, a tab containing the window's name appears along the IDE window's left, right or bottom edge (Fig. 2.10). Clicking the name of an auto-hidden window displays that window (Fig. 2.11). Clicking the name again (or clicking outside) hides the window. To "pin down" a window (that is, to disable auto-hide and keep the window open), click the pin icon. When auto-hide is enabled, the pin icon is horizontal



as shown in Fig. 2.11. When a window is "pinned down," the pin icon is vertical



as shown in Fig. 2.12.

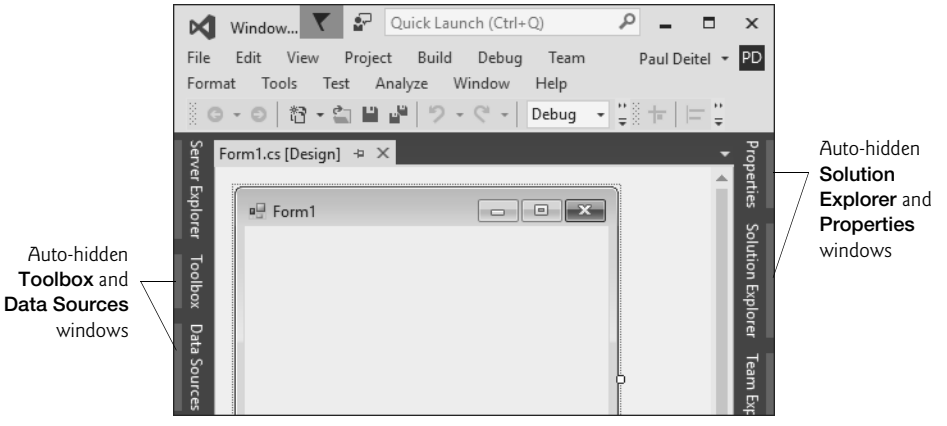


Fig. 2.10 | Auto-hide feature demonstration.

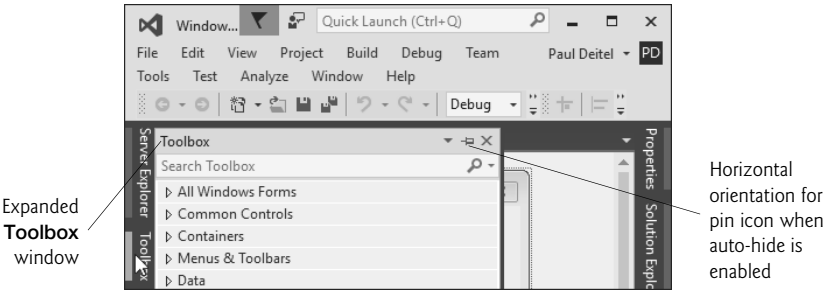


Fig. 2.11 | Displaying the hidden **Toolbox** window when auto-hide is enabled.

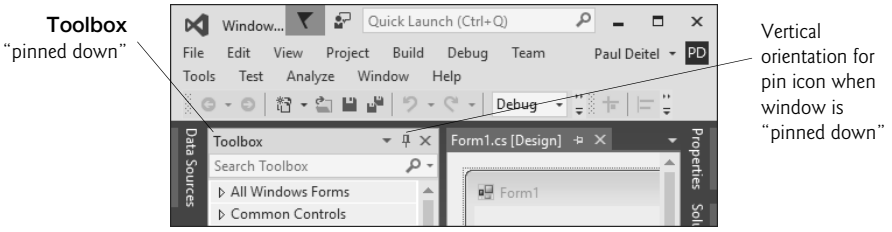


Fig. 2.12 | Disabling auto-hide—"pinning down" a window.

The next few sections present three Visual Studio's windows that you'll use frequently—the **Solution Explorer**, the **Properties** window and the **Toolbox**. These windows display project information and include tools that help you build your apps.

2.4.1 Solution Explorer

The **Solution Explorer** window (Fig. 2.13) provides access to all of a solution's files. If it's not shown in the IDE, select **View > Solution Explorer**. When you open a new or existing solution, the **Solution Explorer** displays the solution's contents.

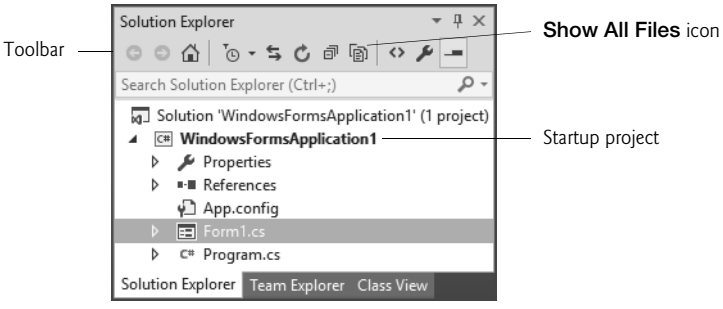


Fig. 2.13 | **Solution Explorer** window showing the **WindowsFormsApplication1** project.

The solution's **startup project** (shown in **bold** in the **Solution Explorer**) is the one that runs when you select **Debug > Start Debugging** (or press *F5*) or select **Debug > Start Without Debugging** (or press *Ctrl* + *F5* key). For a single-project solution like the examples in this book, the startup project is the only project (in this case, **WindowsFormsApplication1**). When you create an app for the first time, the **Solution Explorer** window appears as shown in Fig. 2.13. The Visual C# file that corresponds to the `Form` shown in Fig. 2.4 is named `Form1.cs` (selected in Fig. 2.13). Visual C# files use the `.cs` file-name extension, which is short for "C#."

By default, the IDE displays only files that you may need to edit—other files that the IDE generates are hidden. The **Solution Explorer** window includes a toolbar that contains several icons. Clicking the **Show All Files icon** (Fig. 2.13) displays all the solution's files, including those generated by the IDE. Clicking the arrow to the left of a node *expands* or *collapses* that node. Click the arrow to the left of **References** to display items grouped

under that heading (Fig. 2.14). Click the arrow again to collapse the tree. Other Visual Studio windows also use this convention.

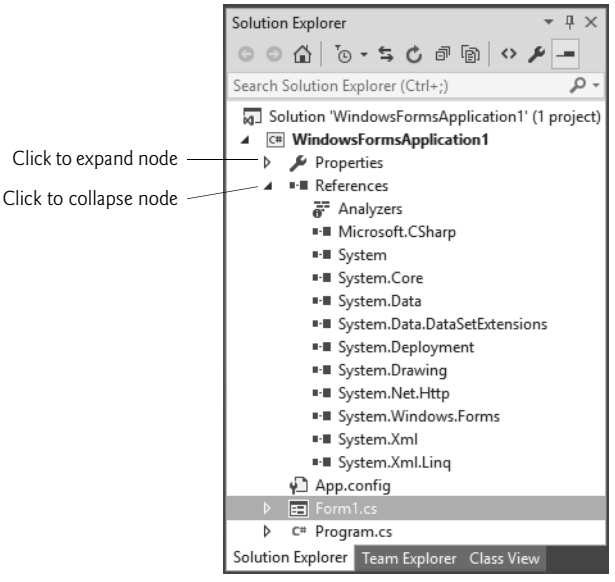


Fig. 2.14 | **Solution Explorer** with the **References** node expanded.

2.4.2 Toolbox

To display the **Toolbox** window, select **View > Toolbox**. The **Toolbox** contains the controls used to customize `Forms` (Fig. 2.15). With visual app development, you can "drag and drop" controls onto the `Form` and the IDE will write the code that creates the controls for you. This is faster and simpler than writing this code yourself. Just as you do not need to know how to build an engine to drive a car, you do not need to know how to build controls to use them. *Reusing* preexisting controls saves time and money when you develop apps. You'll use the **Toolbox** when you create your first app later in the chapter.

The **Toolbox** groups the prebuilt controls into categories—**All Windows Forms**, **Common Controls**, **Containers**, **Menus & Toolbars**, **Data**, **Components**, **Printing**, **Dialogs**, **Reporting**, **WPF Interoperability** and **General** are listed in Fig. 2.15. Again, note the use of arrows for expanding or collapsing a group of controls. We discuss many of the **Toolbox**'s controls and their functionality throughout the book.

2.4.3 Properties Window

If the **Properties** window is not displayed below the **Solution Explorer**, select **View > Properties Window** to display it—if the window is in auto-hide mode, pin down the window by clicking its horizontal pin icon



The **Properties window** contains the properties for the currently selected `Form`, control or file in the IDE. Properties specify information about the `Form` or control, such as its size,

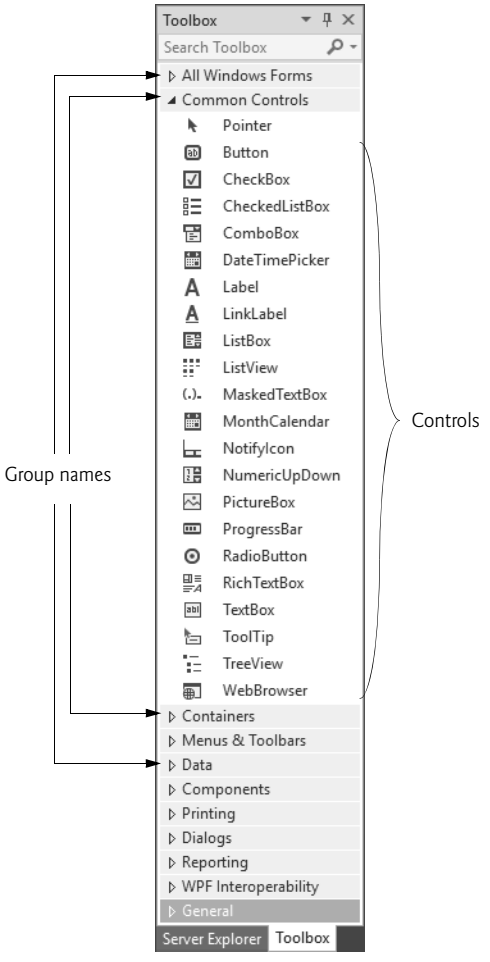


Fig. 2.15 | **Toolbox** window displaying controls for the **Common Controls** group.

color and position. Each `Form` or control has its own set of properties. When you select a property, its description is displayed at the bottom of the **Properties** window.

Figure 2.16 shows `Form1`'s **Properties** window—you can view by clicking anywhere in the **Form1.cs [Design]** window. The left column lists the `Form`'s properties—the right column displays the current value of each property. You can sort the properties either

- *alphabetically* (by clicking the **Alphabetical** icon) or
- *categorically* (by clicking the **Categorized** icon).

Depending on the **Properties** window's size, some properties may be hidden from your view. You can scroll through the list of properties by dragging the scrollbox up or down inside the scrollbar, or by clicking the arrows at the top and bottom of the scrollbar. We show how to set individual properties later in this chapter.

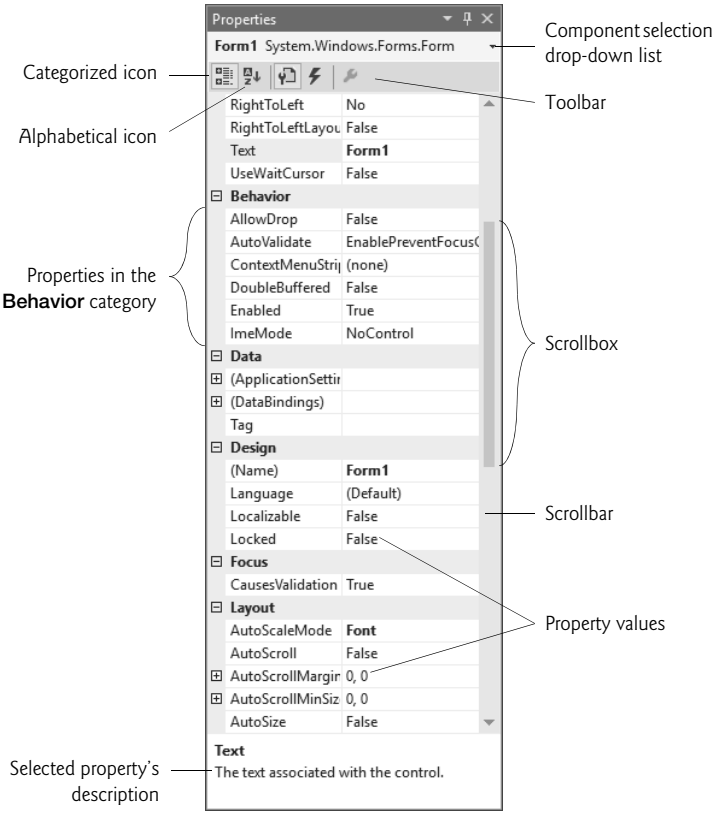


Fig. 2.16 | **Properties** window.

The **Properties** window is crucial to visual app development—it allows you to quickly modify a control's properties and, rather than writing code yourself, lets the IDE write code for you "behind the scenes." You can see which properties are available for modification and, in many cases, can learn the range of acceptable values for a given property. The **Properties** window displays a brief description of the selected property, helping you understand its purpose.

2.5 Help Menu and Context-Sensitive Help

Microsoft provides extensive help documentation via the **Help** menu, which is an excellent way to get information quickly about Visual Studio, Visual C# and more. Visual Studio provides context-sensitive help pertaining to the "current content" (that is, the items around the location of the mouse cursor). To use context-sensitive help, click an item, then press the *F1* key. The help documentation is displayed in a web browser window. To return to the IDE, either close the browser window or select the IDE's icon in your Windows task bar.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image

Next, we create an app that displays the text `"Welcome to C# Programming!"` and an image of the Deitel & Associates bug mascot. The app consists of a `Form` that uses a `Label` and a `PictureBox`. Figure 2.17 shows the final app executing. The app and the bug image are available with this chapter's examples—see the Before You Begin section following the Preface for download instructions. We assume you placed the examples in your user account's `Documents` folder in a subfolder named `examples`.

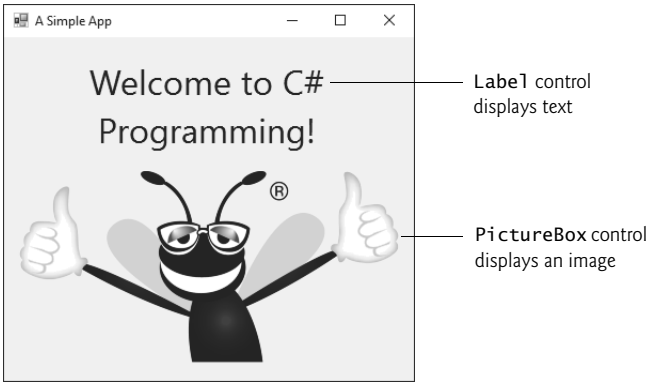


Fig. 2.17 | Simple app executing.

In this example, you won't write any C# code. Instead, you'll use visual app-development techniques. Visual Studio processes your actions (such as mouse clicking, dragging and dropping) to generate app code. Chapter 3 begins our discussion of writing app code. Throughout the book, you'll produce increasingly substantial and powerful apps that will include code written by you and code generated by Visual Studio. The generated code can be difficult for novices to understand—but you'll rarely need to look at it.

Visual app development is useful for building GUI-intensive apps that require a significant amount of user interaction. To create, save, run and terminate this first app, perform the following steps.

Step 1: Closing the Open Project

If the project you were working with earlier in this chapter is still open, close it by selecting **File > Close Solution**.

Step 2: Creating the New Project

To create a new Windows `Forms` app:

1. Select **File > New > Project...** to display the **New Project** dialog (Fig. 2.18).
2. Select **Windows Forms Application**. Name the project **ASimpleApp**, specify the **Location** where you want to save it and click **OK**. We stored the app in the IDE's default location—in your user account's `Documents` folder under the `Visual Studio 2015\Projects`.

As you saw earlier in this chapter, when you first create a new Windows `Forms` app, the IDE opens in **Design** view (that is, the app is being designed and is not executing). The text **Form1.cs [Design]** in the tab containing the `Form` means that we're designing the `Form` *visually* rather than *programmatically*. An asterisk (*) at the end of the text in a tab indicates that you've changed the file and the changes have not yet been saved.

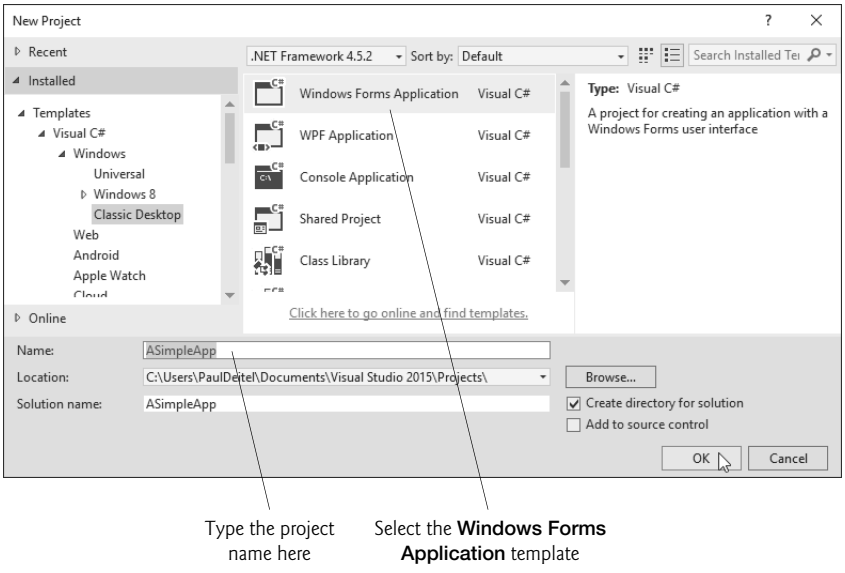


Fig. 2.18 | **New Project** dialog.

Step 3: Setting the Text in the `Form`'s Title Bar

The text in the `Form`'s title bar is determined by the `Form`'s **`Text` property** (Fig. 2.19). If the **Properties** window is not open, select **View > Properties Window** and pin down the window so it doesn't auto hide. Click anywhere in the `Form` to display the `Form`'s properties in the **Properties** window. In the textbox to the right of the `Text` property, type `"A Simple App"`, as in Fig. 2.19. Press the *Enter* key—the `Form`'s title bar is updated immediately (Fig. 2.20).

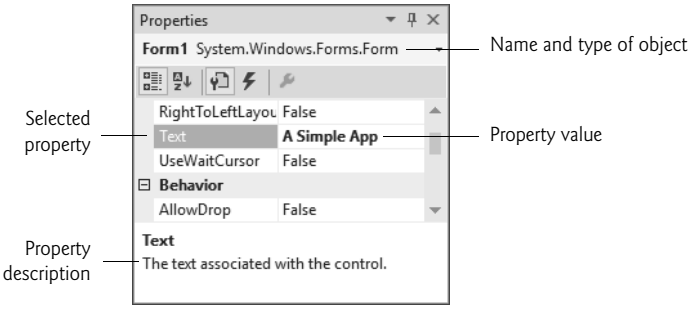


Fig. 2.19 | Setting the `Form`'s `Text` property in the **Properties** window.

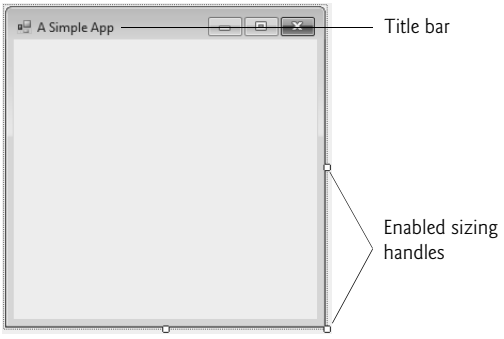


Fig. 2.20 | `Form` with updated title-bar text and enabled sizing handles.

Step 4: Resizing the `Form`

The `Form`'s size is specified in pixels (that is, dots on the screen). By default, a `Form` is 300 pixels wide and 300 pixels tall. You can resize the `Form` by dragging one of its **sizing handles** (the small white squares that appear around the `Form`, as shown in Fig. 2.20). Using the mouse, select the bottom-right sizing handle and drag it down and to the right to make the `Form` larger. As you drag the mouse (Fig. 2.21), the IDE's status bar (at the bottom of the IDE) shows the current width and height in pixels. We set the `Form` to 400 pixels wide by 360 pixels tall. You also can do this via the `Form`'s `Size` property in the **Properties** window.

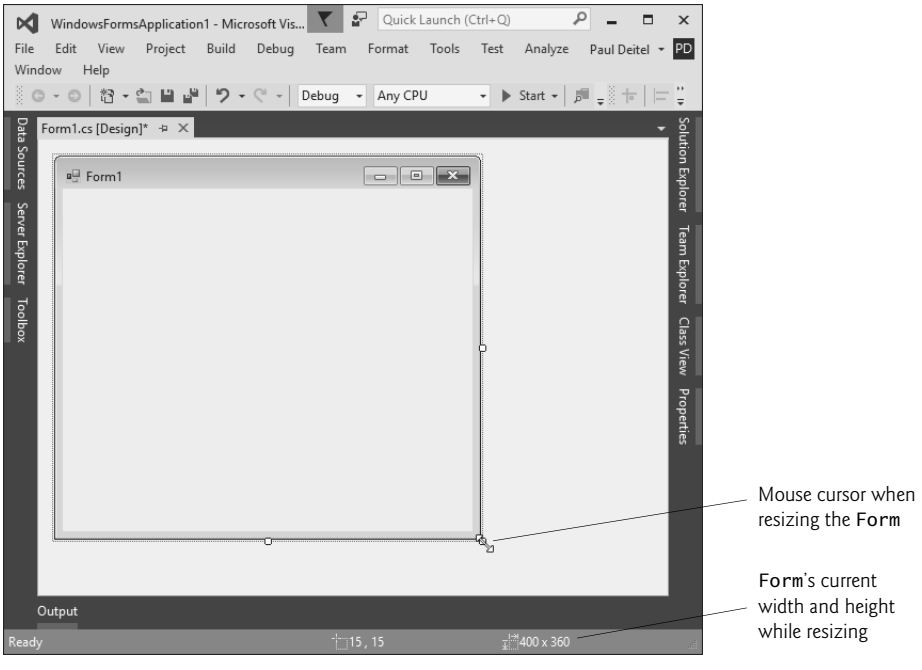


Fig. 2.21 | Resizing the `Form`.

Step 5: Changing the `Form`'s Background Color

The **`BackColor` property** specifies a `Form`'s or control's background color. Clicking `BackColor` in the **Properties** window causes a down-arrow button to appear next to the value of the property (Fig. 2.22). Clicking the down-arrow button displays other options, which vary depending on the property. In this case, the arrow displays tabs for **Custom**, **Web** and **System** (the default). Click the **Custom** tab to display the palette (a grid of colors). Select the box that represents light blue. Once you select the color, the palette closes and the `Form`'s background color changes to light blue (Fig. 2.23).

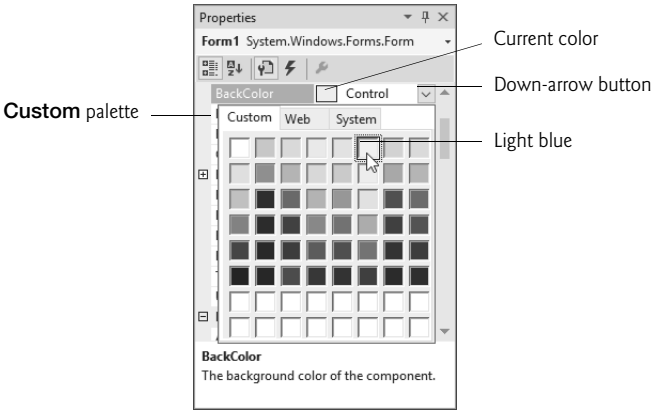


Fig. 2.22 | Changing the `Form`'s `BackColor` property.

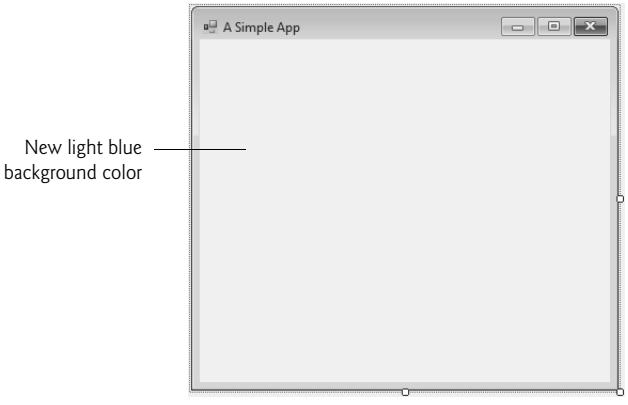


Fig. 2.23 | `Form` with new `BackColor` property applied.

Step 6: Adding a `Label` Control to the `Form`

For the app we're creating in this chapter, the typical controls we use are located in the **Toolbox**'s **Common Controls** group, and also can be found in the **All Windows Forms** group. If the **Toolbox** is *not* already open, select **View > Toolbox** to display the set of controls you'll use for creating your apps. If either group name is collapsed, expand it by clicking the ar-

row to the left of the group name (the **All Windows Forms** and **Common Controls** groups are shown in Fig. 2.15). Next, double click the `Label` control in the **Toolbox** to add a `Label` in the `Form`'s upper-left corner (Fig. 2.24)—each `Label` you add to the `Form` is an object of class `Label` from the .NET Framework Class Library. [*Note:* If the `Form` is behind the **Toolbox**, you may need to hide or pin down the **Toolbox** to see the `Label`.] Although double clicking any **Toolbox** control places the control on the `Form`, you also can "drag" controls from the **Toolbox** to the `Form`—you may prefer dragging the control because you can position it wherever you want. The `Label` displays the text **label1** by default. By default, the `Label`'s `BackColor` is the same as the `Form`'s.

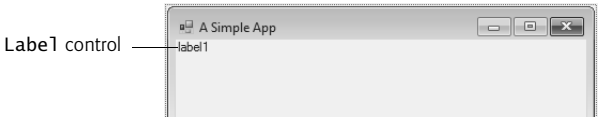


Fig. 2.24 | Adding a `Label` to the `Form`.

Step 7: Customizing the `Label`'s Appearance

Click the `Label`'s text in the `Form` to select it and display its properties in the **Properties** window. The `Label`'s `Text` property determines the text that the `Label` displays. The `Form` and `Label` each have their own `Text` property—`Form`s and controls can have the *same* property names (such as `Text`, `BackColor` etc.) without conflict. Each common properties purpose can vary by control. Perform the following steps:

1. Set the `Label`'s `Text` property to `Welcome to C# Programming!`. The `Label` resizes to fit all the typed text on one line.
2. By default, the **`AutoSize` property** of the `Label` is set to `True` so the `Label` can update its own size to fit all of its text. Set the `AutoSize` property to `False` so that you can change the `Label`'s size, then resize the `Label` (using the sizing handles) so that the text fits.
3. Move the `Label` to the top center of the `Form` by dragging it or by using the keyboard's left and right arrow keys to adjust its position (Fig. 2.25). Alternatively, when the `Label` is selected, you can center it horizontally by selecting **Format > Center In Form > Horizontally**.

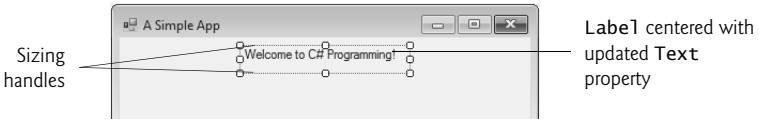


Fig. 2.25 | GUI after the `Form` and `Label` have been customized.

Step 8: Setting the `Label`'s Font Size

To change the font type and appearance of the `Label`'s text:

1. Select the value of the **`Font` property**, which causes an **ellipsis button** to appear next to the value (Fig. 2.26)—you can click this button to display a dialog of op-

tions for the property. Click the ellipsis button to display the **Font** dialog (Fig. 2.27).

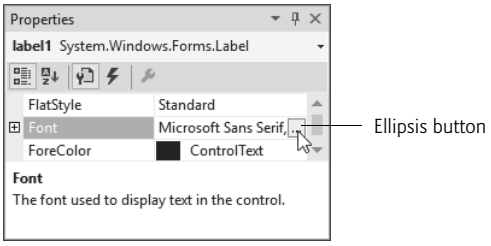


Fig. 2.26 | **Properties** window displaying the `Label`'s `Font` property.

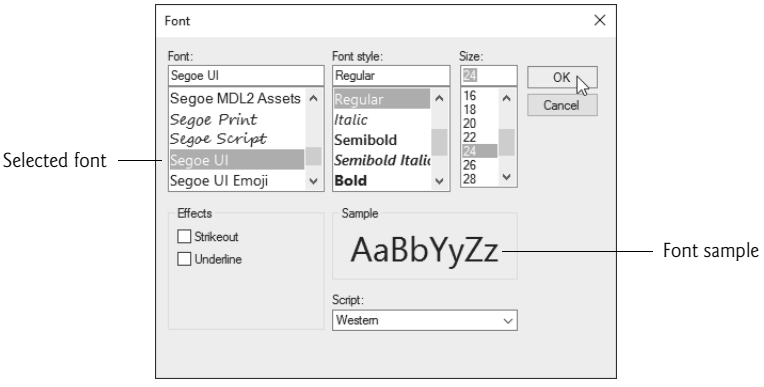


Fig. 2.27 | **Font** dialog for selecting fonts, styles and sizes.

2. You can select the font name (the font options may be different, depending on your system), font style (**Regular**, **Italic**, **Bold**, etc.) and font size (**16**, **18**, **20**, etc.) in this dialog. The **Sample** text shows the selected font settings. Under **Font**, select **Segoe UI**, Microsoft's recommended font for user interfaces. Under **Size**, select **24** points and click **OK**.
3. If the `Label`'s text does not fit on a single line, it *wraps* to the next line. Resize the `Label` so that the words `"Welcome to"` appear on the `Label`'s first line and the words `"C# Programming!"` appear on the second line.
4. Re-center the `Label` horizontally.

Step 9: Aligning the `Label`'s Text

Select the `Label`'s **`TextAlign`** property, which determines how the text is aligned within the `Label`. A three-by-three grid of buttons representing alignment choices is displayed (Fig. 2.28). The position of each button corresponds to where the text appears in the `Label`. For this app, set the `TextAlign` property to `MiddleCenter` in the three-by-three grid—this selection centers the text horizontally and vertically within the `Label`. The other `TextAlign` values, such as `TopLeft`, `TopRight`, and `BottomCenter`, can be used to position

the text anywhere within a `Label`. Certain alignment values may require that you resize the `Label` to fit the text better.

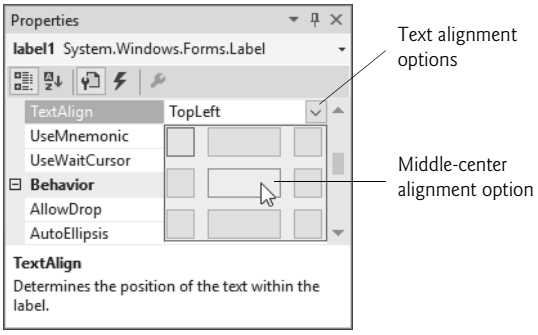


Fig. 2.28 | Centering the `Label`'s text.

Step 10: Adding a `PictureBox` to the `Form`

The `PictureBox` control displays images. Locate the `PictureBox` in the **Toolbox** (Fig. 2.15) and double click it to add it to the `Form`—each `PictureBox` you add to the `Form` is an object of class `PictureBox` from the .NET Framework Class Library. When the `PictureBox` appears, move it underneath the `Label`, either by dragging it or by using the arrow keys (Fig. 2.29).

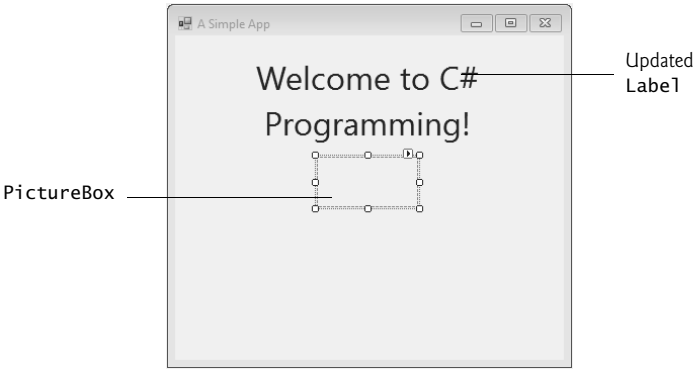


Fig. 2.29 | Inserting and aligning a `PictureBox`.

Step 11: Inserting an Image

Click the `PictureBox` to display its properties in the **Properties** window (Fig. 2.30), then:

1. Locate and select the **`Image` property**, which displays a preview of the selected image or **(none)** if no image is selected.
2. Click the ellipsis button to display the **Select Resource dialog** (Fig. 2.31), which is used to import files, such as images, for use in an app.

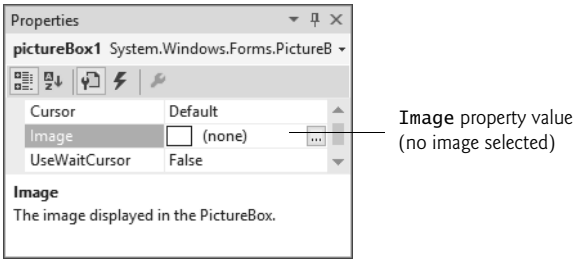


Fig. 2.30 | `Image` property of the `PictureBox`.

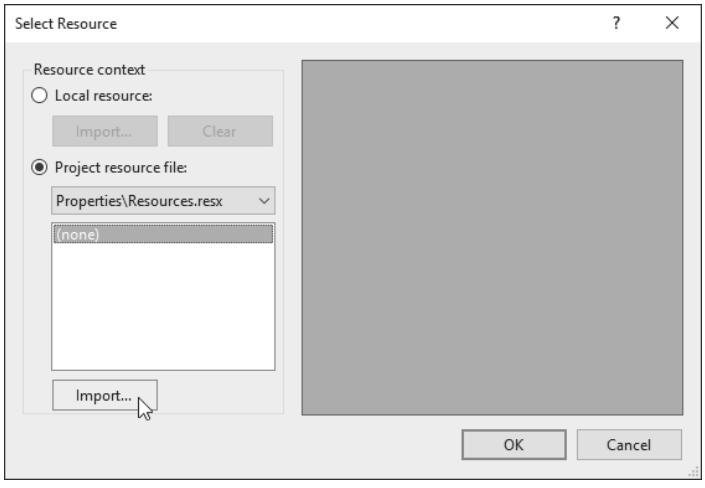


Fig. 2.31 | **Select Resource** dialog to select an image for the `PictureBox`.

3. Click the **Import...** button to browse for an image, select the image file and click **OK** to add it to your project. We used `bug.png` from this chapter's examples folder. Supported image formats include PNG (Portable Network Graphics), GIF (Graphic Interchange Format), JPEG (Joint Photographic Experts Group) and BMP (Windows bitmap). Depending image's size, it's possible that only a portion of the image will be previewed in the **Select Resource** dialog—you can resize the dialog to see more of the image (Fig. 2.32). Click **OK** to use the image.
4. To scale the image to fit in the `PictureBox`, change the **`SizeMode` property** to **`StretchImage`** (Fig. 2.33). Resize the `PictureBox`, making it larger (Fig. 2.34), then re-center the `PictureBox` horizontally.

Step 12: Saving the Project

Select **File > Save All** to save the entire solution. The solution file (which has the filename extension `.sln`) contains the name and location of its project, and the project file (which has the filename extension `.csproj`) contains the names and locations of all the files in the project. If you want to reopen your project at a later time, simply open its `.sln` file.

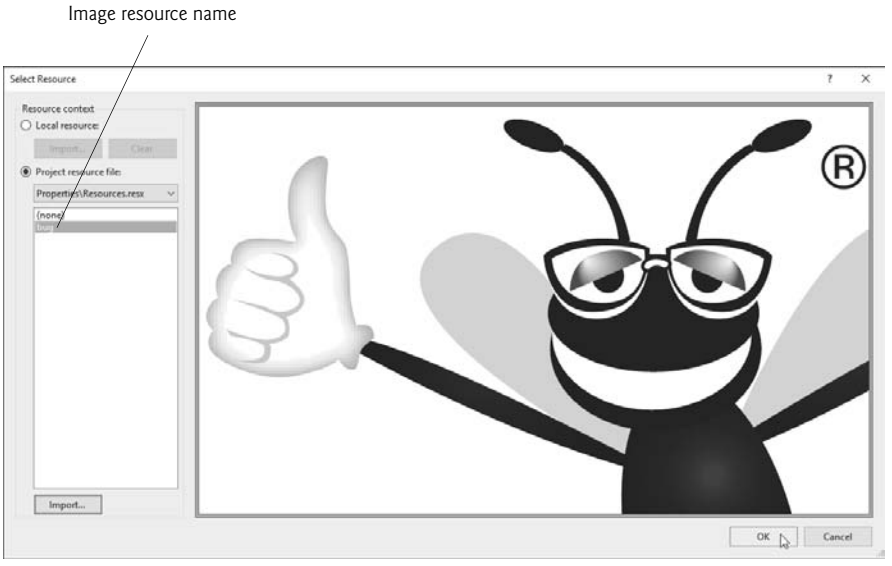


Fig. 2.32 | **Select Resource** dialog displaying a preview of selected image.

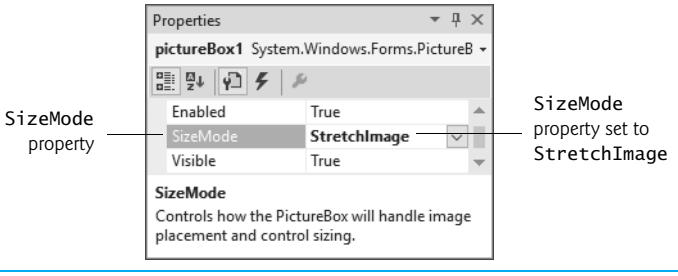


Fig. 2.33 | Scaling an image to the size of the `PictureBox`.

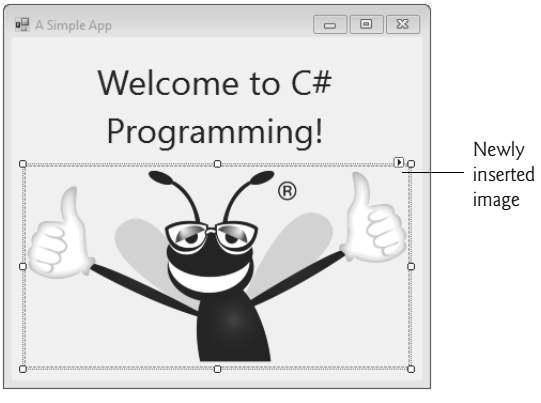


Fig. 2.34 | `PictureBox` displaying an image.

Step 13: Running the Project
Recall that up to this point we have been working in the IDE design mode (that is, the app being created is not executing). In **run mode**, the app is executing, and you can interact with only a few IDE features—features that are not available are disabled (grayed out). Select **Debug > Start Debugging** to execute the app (or press the *F5* key). The IDE enters run mode and displays "**(Running)**" next to the app's name in the IDE's title bar. Figure 2.35 shows the running app, which appears in its own window outside the IDE.

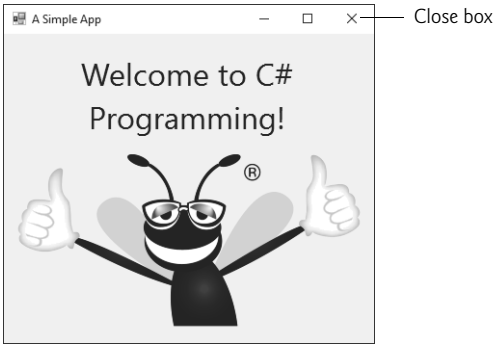


Fig. 2.35 | IDE in run mode, with the running app in the foreground.

Step 14: Terminating the App
You can terminate the app by clicking its close box



in the top-right corner of the running app's window. This action stops the app's execution and returns the IDE to design mode. You also can select **Debug > Stop Debugging** to terminate the app.

2.7 Wrap-Up

In this chapter, we introduced key features of the Visual Studio IDE. You visually designed a working Visual C# app without writing any code. Visual C# app development is a mixture of the two styles—visual app development allows you to develop GUIs easily and avoid tedious GUI programming and "conventional" programming (which we introduce in Chapter 3) allows you to specify the behavior of your apps.

You created a Visual C# Windows `Forms` app with one `Form`. You worked with the IDE's **Solution Explorer**, **Toolbox** and **Properties** windows, which are essential to developing Visual C# apps. We also demonstrated context-sensitive help, which displays help topics related to selected controls or text.

You used visual app development to design an app's GUI by adding a `Label` and a `PictureBox` control onto a `Form`. You used the **Properties** window to set a `Form`'s `Text` and `BackColor` properties. You learned that `Label` controls display text and that `PictureBox`es display images. You displayed text in a `Label` and added an image to a `PictureBox`. You also worked with the `Label`'s `AutoSize`, `TextAlign` and `Font` properties and the `PictureBox`'s `Image` and `SizeMode` properties.

In the next chapter, we discuss "nonvisual," or "conventional," programming—you'll create your first apps with C# code that you write, instead of having Visual Studio write the code. You'll learn memory concepts and write code that displays information on the screen, receives inputs from the user at the keyboard, performs arithmetic operations and makes decisions.

2.8 Web Resources

Please take a moment to visit each of these sites.

`https://www.visualstudio.com/`
The home page for Microsoft Visual Studio. The site includes news, documentation, downloads and other resources.

`https://social.msdn.microsoft.com/Forums/vstudio/en-US/home?forum=csharpgeneral`
This site provides access to the Microsoft Visual C# forums, which you can use to get your Visual C# language and IDE questions answered.

`https://msdn.microsoft.com/magazine/default.aspx`
This is the Microsoft Developer Network Magazine site. This site provides articles and code on many Visual C# and .NET app development topics. There is also an archive of past issues.

`http://stackoverflow.com/`
In addition to the Microsoft forums, StackOverflow is an excellent site for getting your programming questions answered for most programming languages and technologies.

Summary

Section 2.1 Introduction

- Visual Studio is Microsoft's Integrated Development Environment (IDE) for creating, running and debugging apps written in a variety of .NET programming languages.
- Creating simple apps by dragging and dropping predefined building blocks into place is called visual app development.

Section 2.2 Overview of the Visual Studio Community 2015 IDE

- The **Start Page** contains links to Visual Studio IDE resources and web-based resources.
- A project is a group of related files that compose a app.
- Visual Studio organizes apps into projects and solutions. A solution may contain one or more projects.
- Dialogs are windows that facilitate user–computer communication.
- Visual Studio provides templates for the project types you can create.
- A `Form` represents the main window of the Windows `Forms` app that you're creating.
- Collectively, the `Form` and controls constitute the app's graphical user interface (GUI), which is the visual part of the app with which the user interacts.

Section 2.3 Menu Bar and Toolbar

- Commands for managing the IDE and for developing, maintaining and executing apps are contained in the menus, which are located on the menu bar.

- Menus contain groups of commands (menu items) that, when selected, cause the IDE to perform actions (for example, open a window, save a file, print a file and execute an app).
- Tool tips help you become familiar with the IDE's features.

Section 2.4 Navigating the Visual Studio IDE

- The **Solution Explorer** window lists all the files in the solution.
- The **Toolbox** contains controls for customizing `Forms`.
- By using visual app development, you can place predefined controls onto the `Form` instead of writing the code yourself.
- Clicking an auto-hidden window's name opens that window. Clicking the name again hides it. To "pin down" a window (that is, to disable auto-hide), click its pin icon.
- The **Properties** window displays the properties for a `Form`, control or file (in **Design** view). Properties are information about a `Form` or control, such as size, color and position. The **Properties** window allows you to modify `Forms` and controls visually, without writing code.
- Each control has its own set of properties. The left column of the **Properties** window shows the property names and the right column displays the property values. This window's toolbar contains options for organizing properties alphabetically when the **Alphabetical** icon is selected or categorically (for example, **Appearance**, **Behavior**, **Design**) when the **Categorized** icon is selected.

Section 2.5 Help Menu and Context-Sensitive Help

- Extensive help documentation is available via the **Help** menu.
- Context-sensitive help brings up a list of relevant help articles. To use context-sensitive help, select an item and press the *F1* key.

Section 2.6 Visual Programming: Creating a Simple App that Displays Text and an Image

- Visual C# app development usually involves a combination of writing a portion of the app code and having Visual Studio generate the remaining code.
- The text that appears at the top of the `Form` (the title bar) is specified in the `Form`'s `Text` property.
- To resize the `Form`, click and drag one of the `Form`'s enabled sizing handles (the small squares around the `Form`). Enabled sizing handles appear as white boxes.
- The `BackColor` property specifies the background color of a `Form`. The `Form`'s background color is the default background color for any controls added to the `Form`.
- Double clicking any **Toolbox** control icon places a control of that type on the `Form`. Alternatively, you can drag and drop controls from the **Toolbox** to the `Form`.
- The `Label`'s `Text` property determines the text (if any) that the `Label` displays. The `Form` and `Label` each have their own `Text` property.
- A property's ellipsis button, when clicked, displays a dialog containing additional options.
- In the **Font** dialog, you can select the font for text in the user interface.
- The `TextAlign` property determines how the text is aligned within a `Label`'s boundaries.
- The `PictureBox` control displays images. The `Image` property specifies the image to displayed.
- An app that is in design mode is not executing.
- In run mode, the app is executing—you can interact with only a few IDE features.
- When designing an app visually, the name of the Visual C# file appears in the project tab, followed by **[Design]**.
- Terminate an app's execution by clicking its close box.

Terminology

active tab
Alphabetical icon
auto-hide
`AutoSize` property of `Label`
`BackColor` property of `Form`
Categorized icon
component selection drop-down list
context-sensitive help
control
Custom tab
Design view
dialog
dragging
ellipsis button
Font dialog
`Font` property of `Label`
`Form`
graphical user interface (GUI)
Help menu
icon
`Image` property of `PictureBox`
`Label`
New Project dialog
palette
`PictureBox`
project
Properties window
property of a `Form` or control
run mode
scrollbar
scrollbox
Select Resource dialog
Show All Files icon
`SizeMode` property of `PictureBox`
sizing handle
solution
Solution Explorer in Visual Studio
Start Page
startup project
`StretchImage` value
templates for projects
`Text` property
`TextAlign` property of `Label`
tool tip
toolbar
visual app development
Visual Studio
Windows `Forms` app

Self-Review Exercises

2.1 Fill in the blanks in each of the following statements:
a) The technique of _____ allows you to create GUIs without writing any code.
b) A(n) _____ is a group of one or more projects that collectively form a Visual C# app.
c) The _____ feature hides a window in the IDE.
d) A(n) _____ appears when the mouse pointer hovers over an icon.
e) The _____ window allows you to browse solution files.
f) The properties in the **Properties** window can be sorted _____ or _____.
g) A `Form`'s _____ property specifies the text displayed in the `Form`'s title bar.
h) The _____ contains the controls that you can add to a `Form`.
i) _____ displays relevant help articles, based on the current context.
j) The _____ property specifies how text is aligned within a `Label`'s boundaries.

2.2 State whether each of the following is *true* or *false*. If *false*, explain why.
a) ✕ toggles auto-hide for a window.
b) The toolbar icons represent various menu commands.
c) The toolbar contains icons that represent controls you can drag onto a `Form`.
d) Both `Forms` and `Labels` have a title bar.
e) Control properties can be modified only by writing code.
f) `PictureBoxes` typically display images.
g) Visual C# files use the file extension `.csharp`.
h) A `Form`'s background color is set using the `BackColor` property.

Answers to Self-Review Exercises

2.1 a) visual app development. b) solution. c) auto-hide. d) tool tip. e) **Solution Explorer**. f) alphabetically, categorically. g) `Text`. h) **Toolbox**. i) context-sensitive help. j) `TextAlign`.

2.2 a) False. The pin icon ( ) toggles auto-hide. ✕ closes a window. b) True. c) False. The **Toolbox** contains icons that represent such controls. d) False. `Forms` have a title bar but `Labels` do not (although they do have `Label` text). e) False. Control properties can be modified using the **Properties** window. f) True. g) False. Visual C# files use the file extension `.cs`. h) True.

Exercises

2.3 Fill in the blanks in each of the following statements:

a) When an ellipsis button is clicked, a(n) _____ is displayed.
b) Using _____ help immediately displays a relevant help article.
c) When you select a property, its description is displayed at the bottom of the _____ window.
d) The _____ property specifies which image a `PictureBox` displays.
e) The _____ menu contains commands for arranging and displaying windows.

2.4 State whether each of the following is *true* or *false*. If *false*, explain why.

a) You can add a control to a `Form` by double clicking its control icon in the **Toolbox**.
b) The `Form`, `Label` and `PictureBox` have identical properties.
c) If your machine is connected to the Internet, you can browse websites from the Visual Studio IDE.
d) Visual C# app developers usually create complex apps without writing any code.
e) Clicking the arrow to the left of a node *expands* or *collapses* that node.

2.5 Some features that appear throughout Visual Studio perform similar actions in different contexts. Explain and give examples of how the ellipsis buttons, down-arrow buttons and tool tips act in this manner. Why do you think the Visual Studio IDE was designed this way?

2.6 Briefly describe each of the following terms:

a) Project
b) IDE
c) Debug
d) Properties window
e) Dialogs
f) Visual Studio

Note Regarding Exercises 2.7–2.11

In the following exercises, you're asked to create GUIs using controls that we have not yet discussed in this book. These exercises give you practice with visual app development only—the apps do not perform any actions. You place controls from the **Toolbox** on a `Form` to familiarize yourself with what each control looks like. If you follow the step-by-step instructions, you should be able to recreate the sample GUIs we show.

2.7 ***(Stop Watch GUI)*** Create the GUI for the Stop Watch as shown in Fig. 2.36.

a) ***Manipulating the Form's properties.*** Change the Text property of the Form to Stop_Watch_Form.
b) ***Adding a MenuStrip control to the Form.*** Add a MenuStrip to the Form. After inserting the MenuStrip, add items by clicking the **Type Here** section, typing a menu name (for example, **Start**, **Stop**, **Lap**, **Reset** and **Exit**) and then pressing *Enter*.

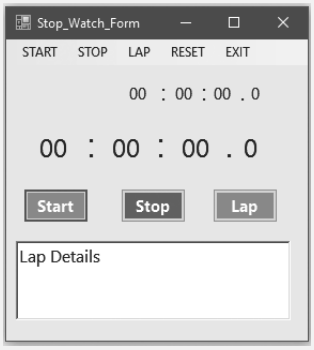


Fig. 2.36 | Notepad GUI.

c) ***Adding Labels to the Form.*** Add 7 Labels each for the display of main stop watch timer and for the Lap timer. Set the **Font** property as **Segoe UI**, **18pt**, **style=Regular** for main timer and **Segoe UI**, **11pt**, **style=Regular** for lap timer. Set the Text properties accordingly.

d) ***Adding Buttons to the Form.*** Add Three Buttons with Text property **Start**, **Stop** and **Lap**. Change their BackColor and ForeColor properties as shown.

e) ***Adding a RichTextBox to the Form.*** Drag this control onto the Form. Use the sizing handles to resize and position the RichTextBox as shown in Fig. 2.36. Change the Text property to Lap Details.

2.8 ***(Login Form GUI)*** Create the GUI for the login screen as shown in Fig. 2.37.



Fig. 2.37 | Calendar and appointments GUI.

a) ***Manipulating the Form's properties.*** Change the Text property of the Form to User Login Form. Set the BackColor property as shown in figure 2.37.

b) ***Adding Labels to the Form.*** Add three Labels to the Form. Set the Label's Text, Font and BackColor properties to match Fig. 2.37. Use Bookman Old Style, 14.25pt font size for User ID and Password.

c) ***Adding TextBoxes to the Form.*** Add two TextBox controls to the Form, for entering userID and password. Set the suitable BackColor and '*' as the PasswordChar property.

d) ***Adding Buttons to the Form.*** Add Three Buttons with Text property New **User**, **Login** and **Forgot password**. Change their BackColor and ForeColor properties as shown.

2.9 ***(Calculator GUI)*** Create the GUI for the calculator as shown in Fig. 2.38.

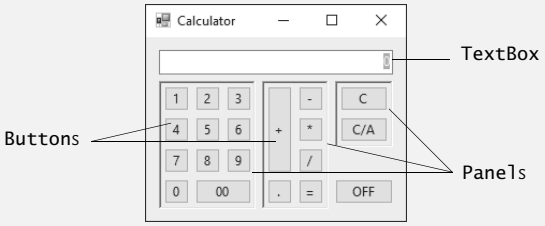


Fig. 2.38 | Calculator GUI.

a) ***Manipulating the `Form`'s properties.*** Change the `Text` property of the `Form` to `Calculator`. Change the `Font` property to 9pt `Segoe UI`. Change the `Size` property of the `Form` to `258, 210`.

b) ***Adding a `TextBox` to the Form.*** Set the `TextBox`'s `Text` property in the **Properties** window to `0`. Stretch the `TextBox` and position it as shown in Fig. 2.38. Set the `TextAlign` property to `Right`—this right aligns text displayed in the `TextBox`.

c) ***Adding the first `Panel` to the Form.*** `Panel` controls are used to group other controls. Add a `Panel` to the `Form`. Change the `Panel`'s `BorderStyle` property to `Fixed3D` to make the inside of the `Panel` appear recessed. Change the `Size` property to `90, 120`. This `Panel` will contain the calculator's numeric keys.

d) ***Adding the second `Panel` to the Form.*** Change the `Panel`'s `BorderStyle` property to `Fixed3D`. Change the `Size` property to `62, 120`. This `Panel` will contain the calculator's operator keys.

e) ***Adding the third (and last) `Panel` to the Form.*** Change the `Panel`'s `BorderStyle` property to `Fixed3D`. Change the `Size` property to `54, 62`. This `Panel` contains the calculator's **C** (clear) and **C/A** (clear all) keys.

f) ***Adding `Buttons` to the Form.*** There are 20 `Buttons` on the calculator. Add a `Button` to the `Panel` by dragging and dropping it on the `Panel`. Change the `Text` property of each `Button` to the calculator key it represents. The value you enter in the `Text` property will appear on the face of the `Button`. Finally, resize the `Buttons`, using their `Size` properties. Each `Button` labeled `0`–`9`, `*`, `/`, `-`, `=` and `.` should have a size of `23, 23`. The **00** `Button` has size `52, 23`. The **OFF** `Button` has size `54, 23`. The **+** `Button` is sized `23, 81`. The **C** (clear) and **C/A** (clear all) `Buttons` are sized `44, 23`.

2.10 ***(Alarm Clock GUI)*** Create the GUI for the alarm clock as shown in Fig. 2.39.

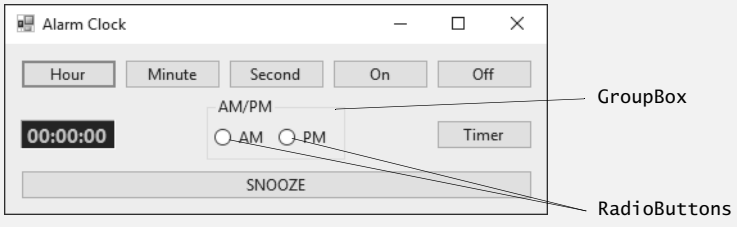


Fig. 2.39 | Alarm clock GUI.

a) *Manipulating the `Form`'s properties.* Change the `Text` property of the `Form` to `Alarm Clock`. Change the `Font` property to 9pt `Segoe UI`. Change the `Size` property of the `Form` to `438, 170`.
b) *Adding `Buttons` to the `Form`.* Add seven `Button`s to the `Form`. Change the `Text` property of each `Button` to the appropriate text. Align the `Button`s as shown.
c) *Adding a `GroupBox` to the `Form`.* `GroupBox`es are like `Panel`s, except that `GroupBox`es display a title. Change the `Text` property to `AM/PM`, and set the `Size` property to `100, 50`. Center the `GroupBox` horizontally on the `Form`.
d) *Adding `AM/PM RadioButtons` to the `GroupBox`.* Place two `RadioButton`s in the `GroupBox`. Change the `Text` property of one `RadioButton` to `AM` and the other to `PM`. Align the `RadioButton`s as shown.
e) *Adding the time `Label` to the `Form`.* Add a `Label` to the `Form` and change its `Text` property to `00:00:00`. Change the `BorderStyle` property to `Fixed3D` and the `BackColor` to `Black`. Use the `Font` property to make the time bold and 12pt. Change the `ForeColor` to `Silver` (located in the **Web** tab) to make the time stand out against the black background. Position the `Label` as shown.

2.11 *(Radio GUI)* Create the GUI for the radio as shown in Fig. 2.40. [*Note:* The image used in this exercise is located in this chapter's examples folder.]

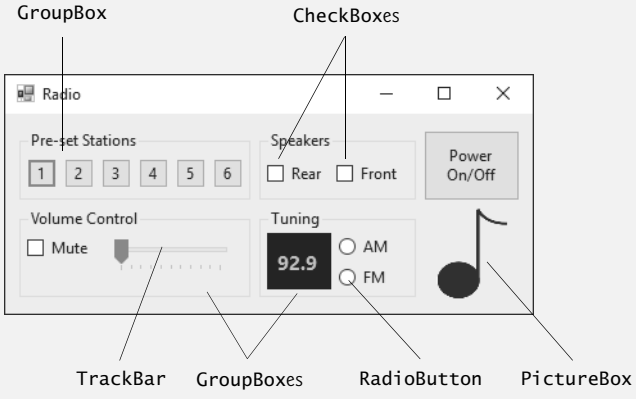


Fig. 2.40 | Radio GUI.

a) *Manipulating the `Form`'s properties.* Change the `Font` property to 9pt `Segoe UI`. Change the `Form`'s `Text` property to `Radio` and the `Size` to `427, 194`.
b) *Adding the **Pre-set Stations** `GroupBox` and `Buttons`.* Set the `GroupBox`'s `Size` to `180, 55` and its `Text` to `Pre-set Stations`. Add six `Button`s to the `GroupBox`. Set each one's `Size` to `23, 23`. Change the `Button`s' `Text` properties to `1`, `2`, `3`, `4`, `5`, `6`, respectively.
c) *Adding the **Speakers** `GroupBox` and `CheckBoxes`.* Set the `GroupBox`'s `Size` to `120, 55` and its `Text` to `Speakers`. Add two `CheckBox`es to the `GroupBox`. Set the `Text` properties for the `CheckBox`es to `Rear` and `Front`.
d) *Adding the **Power On/Off** `Button`.* Add a `Button` to the `Form`. Set its `Text` to `Power On/Off` and its `Size` to `75, 55`.
e) *Adding the **Volume Control** `GroupBox`, the **Mute** `CheckBox` and the **Volume** `TrackBar`.* Add a `GroupBox` to the `Form`. Set its `Text` to `Volume Control` and its `Size` to `180, 70`. Add a `CheckBox` to the `GroupBox`. Set its `Text` to `Mute`. Add a `TrackBar` to the `GroupBox`.

f) *Adding the* **Tuning** `GroupBox`*, the radio station* `Label` *and the* **AM/FM** `RadioButtons`*.* Add a `GroupBox` to the `Form`. Set its `Text` to `Tuning` and its `Size` to `120, 70`. Add a `Label` to the `GroupBox`. Set its `AutoSize` to `False`, its `Size` to `50, 44`, its `BackColor` to `Black`, its `ForeColor` to `Silver`, its font to 12pt bold and its `TextAlign` to `MiddleCenter`. Set its `Text` to `92.9`. Place the `Label` as shown in the figure. Add two `RadioButtons` to the `GroupBox`. Set the `Text` of one to `AM` and of the other to `FM`.

g) ***Adding the image.*** Add a `PictureBox` to the `Form`. Set its `SizeMode` to `StretchImage` and its `Size` to `55, 70`. Set the `Image` property to `MusicNote.gif` (located in this chapter's examples folder).